

```

/*
 * =====
 * EECS 348 Assignment 2
 *
 * Description:
 * C program that prioritizes emails for a CEO using a Max Heap
 * implemented from scratch as a list-based priority queue.
 *
 * Inputs:
 * Commands from a text file or standard input:
 * EMAIL <sender>,<subject>,<date>
 * NEXT
 * READ
 * COUNT
 *
 * Outputs:
 * Display next email, unread email count, and status messages.
 *
 * Collaborators:
 * None
 *
 * Other Sources:
 * M365 Copilot (used as GenAI source)
 *
 * Author:
 * Your Name
 *
 * Creation Date:
 * 09/17/2026
 *
 * Revision Date:
 * 09/17/2026
 * =====
 */

```

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>

```

```

#define MAX_HEAP_SIZE 1000
#define MAX_SUBJECT 200

```

```

/* Structure representing an email */
typedef struct
{
    char sender[30];
    char subject[MAX_SUBJECT];
    int month;
    int day;
    int year;
}

```

```

} Email;

/* Heap storage */
Email heap[MAX_HEAP_SIZE];
int heapSize = 0;

/*-----*/
/* Convert sender category into priority value */
/* Higher value means higher priority */
/*-----*/
int getPriority(char *sender)
{
    if (strcmp(sender, "Boss") == 0)
        return 5;
    if (strcmp(sender, "Subordinate") == 0)
        return 4;
    if (strcmp(sender, "Peer") == 0)
        return 3;
    if (strcmp(sender, "ImportantPerson") == 0)
        return 2;
    return 1;
}

/*-----*/
/* Convert date into comparable integer */
/*-----*/
long long dateValue(Email e)
{
    return ((long long)e.year * 10000) +
           ((long long)e.month * 100) +
           e.day;
}

/*-----*/
/* Compare two emails */
/* Returns positive if a > b */
/*-----*/
int compareEmails(Email a, Email b)
{
    int priorityA = getPriority(a.sender);
    int priorityB = getPriority(b.sender);

    if (priorityA != priorityB)
        return priorityA - priorityB;

    long long dateA = dateValue(a);
    long long dateB = dateValue(b);

    if (dateA > dateB)
        return 1;

```

```

        if (dateA < dateB)
            return -1;

    return 0;
}

/*-----*/
/* Swap two heap entries */
/*-----*/
void swap(Email *a, Email *b)
{
    Email temp = *a;
    *a = *b;
    *b = temp;
}

/*-----*/
/* Move element upward to restore heap property */
/*-----*/
void heapifyUp(int index)
{
    while (index > 0)
    {
        int parent = (index - 1) / 2;

        if (compareEmails(heap[index], heap[parent]) > 0)
        {
            swap(&heap[index], &heap[parent]);
            index = parent;
        }
        else
        {
            break;
        }
    }
}

/*-----*/
/* Move element downward to restore heap property */
/*-----*/
void heapifyDown(int index)
{
    while (1)
    {
        int left = 2 * index + 1;
        int right = 2 * index + 2;
        int largest = index;

        if (left < heapSize &&
            compareEmails(heap[left], heap[largest]) > 0)

```

```

        {
            largest = left;
        }

        if (right < heapSize &&
            compareEmails(heap[right], heap[largest]) > 0)
        {
            largest = right;
        }

        if (largest != index)
        {
            swap(&heap[index], &heap[largest]);
            index = largest;
        }
        else
        {
            break;
        }
    }
}

/*-----*/
/* Insert email into max heap */
/*-----*/
void insertEmail(Email email)
{
    if (heapSize >= MAX_HEAP_SIZE)
    {
        printf("Heap is full.\n");
        return;
    }

    heap[heapSize] = email;
    heapifyUp(heapSize);
    heapSize++;
}

/*-----*/
/* Return highest priority email */
/*-----*/
Email peekEmail()
{
    return heap[0];
}

/*-----*/
/* Remove highest priority email */
/*-----*/
void removeEmail()

```

```

{
    if (heapSize == 0)
        return;

    heap[0] = heap[heapSize - 1];
    heapSize--;

    if (heapSize > 0)
    {
        heapifyDown(0);
    }
}

/*-----*/
/* Display highest priority email */
/*-----*/
void nextEmail()
{
    if (heapSize == 0)
    {
        printf("No emails to read.\n");
        return;
    }

    Email e = peekEmail();

    printf("Next email:\n");
    printf("Sender: %s\n", e.sender);
    printf("Subject: %s\n", e.subject);
    printf("Date: %02d-%02d-%04d\n\n",
           e.month,
           e.day,
           e.year);
}

/*-----*/
/* Count emails */
/*-----*/
void countEmails()
{
    printf("There are %d emails to read.\n",
           heapSize);
}

/*-----*/
/* Parse EMAIL command */
/*-----*/
void processEmail(char *line)
{
    Email e;

```

```

char sender[30];
char subject[MAX_SUBJECT];
char date[20];

sscanf(line,
        "EMAIL %29[^,],%199[^,],%19s",
        sender,
        subject,
        date);

strcpy(e.sender, sender);
strcpy(e.subject, subject);

sscanf(date,
        "%d-%d-%d",
        &e.month,
        &e.day,
        &e.year);

insertEmail(e);
}

/*-----*/
/* Main program */
/*-----*/
int main()
{
    char line[500];

    while (fgets(line, sizeof(line), stdin))
    {
        line[strcspn(line, "\n")] = '\0';

        if (strncmp(line, "EMAIL", 5) == 0)
        {
            processEmail(line);
        }
        else if (strcmp(line, "NEXT") == 0)
        {
            nextEmail();
        }
        else if (strcmp(line, "READ") == 0)
        {
            removeEmail();
        }
        else if (strcmp(line, "COUNT") == 0)
        {
            countEmails();
        }
    }
}

```

```
    }  
    return 0;  
}
```